

UNIVERSIDAD MESOAMERICANA

TÉCNICO EN DESARROLLO DE SOFTWARE

CURSO: PROGRAMACION MOVIL

API REST Y SOCKETS

KEVIN SANCHEZ

GUATEMALA, 21 DE MAYO DE 2026

INDICE

1. Conexiones API REST (El modelo de "Pregunta y Respuesta").....	2
En Express (Servidor Normal).....	3
□ Cómo se establece	3
□ Parámetros clave	3
En HAProxy (El Intermediario).....	3
□ Cómo se establece	3
□ Parámetros clave (Configuración).....	3
2. Conexiones Sockets / WebSockets (El modelo de "Llamada Telefónica").....	3
En Express (Servidor Normal).....	4
□ Cómo se establece	4
□ Parámetros clave	4
En HAProxy (El Intermediario).....	4
□ Cómo se establece	4
□ Parámetros clave (Configuración).....	4
REST vs Sockets	5

1. Conexiones API REST (El modelo de "Pregunta y Respuesta")

REST se basa en el protocolo HTTP. Es una conexión **unidireccional y temporal**. El cliente (navegador o app) hace una pregunta, el servidor responde y la conexión se cierra inmediatamente.

En Express (Servidor Normal)

Express maneja esto de forma nativa. No requiere configuraciones extrañas porque HTTP es su hábitat natural.

- **Cómo se establece:** El cliente envía una petición (GET, POST, etc.). Express la recibe a través de un puerto, busca la ruta correspondiente, procesa los datos y envía una respuesta.
- **Parámetros clave:**
 - keep-alive: Es un parámetro en la cabecera HTTP. Le dice al servidor: *"Oye, no cierres la conexión TCP inmediatamente, por si te pido otra cosa en los próximos segundos"*. Esto ahorra tiempo.
 - timeouts: El tiempo máximo que Express esperará para recibir la petición o enviar la respuesta antes de colgar (por defecto suelen ser 2 minutos en Node.js, pero se puede ajustar).

En HAProxy (El Intermediario)

Como HAProxy recibe las cartas primero y luego se las pasa a Express, necesita saber cuánto tiempo guardar la conexión.

- **Cómo se establece:** El cliente le pide algo a HAProxy, HAProxy abre una conexión con Express, le pasa la petición, recibe la respuesta de Express y se la devuelve al cliente.
- **Parámetros clave (Configuración):**
 - timeout connect: Cuánto tiempo espera HAProxy para lograr conectarse con tu servidor de Express. Si Express está caído o saturado y pasa este tiempo, da error.
 - timeout client y timeout server: El tiempo máximo de inactividad permitido para el cliente o el servidor Express antes de cerrar la conexión.

2. Conexiones Sockets / WebSockets (El modelo de "Llamada Telefónica")

A diferencia de REST, los Sockets permiten una conexión **bidireccional y persistente**. El cliente y el servidor hablan en tiempo real. La conexión se abre una vez y se queda abierta hasta que alguien cuelgue.

En Express (Servidor Normal)

Express por sí solo no maneja Sockets de forma nativa; se suele usar una librería como **Socket.io** o ws montada sobre el servidor de Node.js.

- **Cómo se establece:** El cliente envía una petición HTTP especial llamada **"Handshake"** (saludo) con una cabecera que dice Upgrade: websocket. Si el servidor acepta, la conexión HTTP "mutas" a una conexión de Socket.
- **Parámetros clave:**
 - pingTimeout y pingInterval (Heartbeat): Como la conexión es eterna, el servidor envía un "pálpito" (ping) cada ciertos segundos (ej. 25s). Si el cliente no responde (pong), el servidor sabe que el usuario perdió el internet y cierra la conexión para no gastar memoria.

En HAProxy (El Intermediario)

Aquí es donde la cosa se pone interesante. HAProxy tiene que entender que esta conexión **no se debe cerrar** después de la primera respuesta.

- **Cómo se establece:** HAProxy recibe el "Handshake", nota la cabecera Upgrade, y si está configurado correctamente, deja el canal abierto permanentemente entre ese cliente específico y ese servidor Express.
- **Parámetros clave (Configuración):**
 - timeout tunnel: Este es el parámetro más importante para Sockets en HAProxy. Reemplaza los timeouts normales de HTTP. Le dice a HAProxy: *"Esta conexión es un túnel de larga duración. No la cierres a menos que pase X tiempo (pueden ser horas o días) sin que ruede ni un solo bit de datos"*.
 - balance source o sticky sessions: Si tienes varios servidores Express, los Sockets necesitan que el cliente siempre hable con el **mismo** servidor donde abrió la sesión. Este parámetro asegura que el tráfico de un usuario no brinque a otro servidor a mitad de la comunicación.

REST vs Sockets

Diferencias fundamentales:

Característica	API REST (HTTP)	Sockets (WebSockets)
Duración	Corta. Nace, responde y muere.	Larga. Se mantiene abierta todo el tiempo.
Dirección del flujo	Solo el cliente puede iniciar la charla.	Ambos pueden hablar cuando quieran (Tiempo real).
Carga en el Servidor	Ligera en reposo. Consume recursos solo al procesar.	Mayor consumo de memoria (RAM), porque mantener miles de teléfonos descolgados ocupa espacio.
Comportamiento HAProxy	Cierra la conexión rápido para liberar espacio.	Usa un tunnel especial y requiere redirigir al usuario siempre al mismo servidor.
Ideal para...	Guardar un usuario, consultar un catálogo, operaciones estándar.	Chats, notificaciones en vivo, videojuegos, dashboards con datos que cambian cada segundo.